

Cloud Computing - 2025/26

Azure Cloud - Lego



Student Name: Sebastião Rusu

Student ID: 73743

Student Name: Leonardo Gonçalves

Student ID: 73767

Student Name: Ricardo Andrade

Student ID: 73768

Abstract

This project was carried out at university level by NOVA School of Science and Technology (FCT NOVA). The aim of this project is to understand how the services available on cloud computing platforms can be used and implemented to create applications that are scalable, fast and always available.

Index

1. Introduction.....	1
2. Design.....	1
2.1. Overall Architecture.....	1
This central application runs on an Azure App Service, which hosts the REST API developed in Java. This API interact with several services:.....	2
2.2. Main Components.....	2
2.3. Component Interaction and Business Logic.....	3
3. Implementation.....	3
4. Testing and Evaluation.....	5
4.1. Web App.....	6
4.2. Cosmos DB.....	6
4.3. Blob Storage.....	6
4.4. Reddis.....	7
5. Conclusion and Future Works.....	8
5.1. Conclusion.....	8
5.2. Future Works.....	8

1. Introduction

This project focuses on the development of the backend for a lego collection management application, leveraging Microsoft Azure cloud services to achieve scalability, performance, and high availability. The system provides a RESTful API that manages users, media, lego sets, comments and auctions (including bid placement).

Functionally, the backend includes the following main entities:

- **User** - user profile containing ID, name, password hash, photo, and owned lego sets;
- **Media** - lego sets pictures;
- **Lego Set** - lego set information containing ID, name, code, description, and respective picture referencing them via photo ID;
- **Comment** - user comments on lego sets;
- **Auction** - lego set auctions specifying seller, bids made, base price and closing date.

The system uses Azure App Service to host the API, Azure Blob Storage to store media files, and Azure Cosmos DB as a NoSQL database for structured data persistence. Performance is enhanced through Azure Cache for Redis, used to cache frequently accessed data, and Azure Functions, which handle asynchronous and scheduled background tasks. Performance evaluation is conducted with Artillery, enabling the assessment of robustness and scalability under varying load and deployment scenarios.

2. Design

2.1. Overall Architecture

The system follows a layered architecture, fully deployed using Microsoft Azure managed services, ensuring modularity, scalability and reliability.

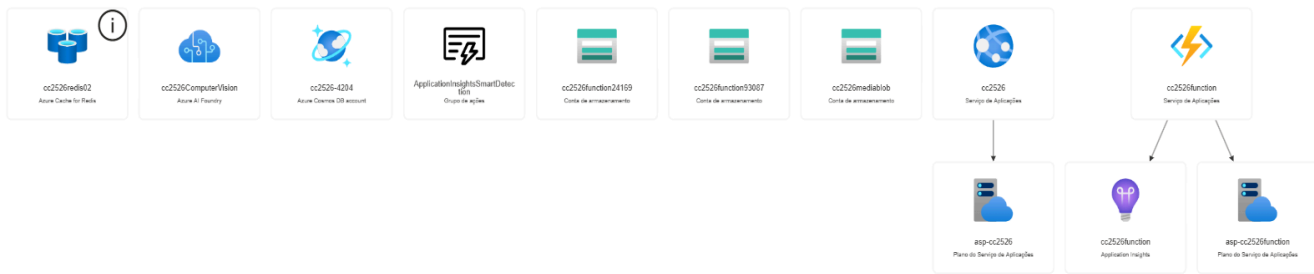


Figure 1. - Implemented infrastructure.

This central application runs on an Azure App Service, which hosts the REST API developed in Java. This API interact with several services:

- **Azure Cosmos DB** - store the users, comments, legosets, comments, lego descriptions and auctions containers;
- **Azure Blob Storage** - repository for multimedia files associated with users and lego sets;
- **Azure Cache for Redis** - distributed caching system to reduce latency and improve performance in frequent read operations;
- **Azure Functions** - function to automatically close expired auctions and function to eliminate expired sessions.
- **Advanced Computation (Computer Vision)** - blob trigger function will run Azure AI Computer Vision service every time a photo is added to our system.

Together, these services work in a complementary way, resulting in a more efficient, scalable, and reliable system.

2.2. Main Components

The system is structured into REST resources, each representing a functional domain of the application. An **user resource** to manage user registration, authentication, and profile retrieval; a **media resource** to handle images interfacing with Blob Storage; a **lego set resource** to manage lego sets including listing and searching operations; a **comment resource** to manage user comments linked to lego sets; an **auction resource** allow creating auctions, retrieving auction data, and placing bids,

ensuring integrity with the involved entities; an **auth resource** to validate user credentials and manage temporary sessions using Redis Cache.

The **close auction function** is executed on a schedule that verifies if auctions are expired, changing their state at Cosmos DB, and the **garbage function**, on schedule too, eliminates expired sessions, as mentioned before.

The **Media Analysis CV pipeline** is executed with a blob trigger, this workflow provides a general description of a media content the moment that is uploaded in the application, and writes the important data in a lego_descriptions container in CosmosDB.

2.3. Component Interaction and Business Logic

The user authenticates through the **auth resource** where the session will be stored in Redis. To create a lego set, an image is uploaded via **media resource** returning the photo ID. The **lego set resource** uses that photo ID to store the set in the Cosmos DB. The **auction resource** creates auctions linked to lego sets and their respective owners and other users place bids that are validated and appended to the auction document. Both functions can be called after to execute their job.

In this way, is observed that the system's business logic relies on three pillars: the user management and secure authentication using hash and Redis sessions; the lego set management maintaining the relation between users and lego sets and linking media stored in Blob Storage; auction and bid management triggering the functions to detect expired auctions marking them as closed.

3. Implementation

The system was implemented following the previously described architecture and integrating different Azure tools. Each REST resource was developed as an independent component, providing modularity and simplifying maintenance.

Security was addressed by storing passwords as SHA-1 hashes, and all entities used Universally Unique Identifiers (UUIDs).

The Azure Cosmos DB database was accessed through a dedicated data access layer, encapsulating all CRUD operations. The Azure Blob Storage was used for image management, separating them from structured data, while Azure Cache for Redis was responsible for session storage and caching of frequently accessed information, with the purpose of reducing response times.

The Azure functions were implemented to handle background tasks automatically. The CloseAuctionsFunction retrieves auctions whose closing date has expired and updates their state to "closed", ensuring temporal consistency in the system.

The GarbageFunction, also scheduled, removes expired user sessions, preventing unnecessary memory usage and maintaining optimal cache performance.

The CV pipeline is activated when an image blob is uploaded to the container, a blob-triggered function calls Azure Computer Vision service to generate description, tags, dominant colors and object counts, then adds that result (description, tags, metadata, timestamps) into Cosmos DB via the CosmosDBLayer. For testing you can visit the app root and use a media_id with GET /rest/media-analysis/<media_id>, there is also a test input field(make sure to test with a valid media_id from the blob storage, you can check these media_id values in the endpoint: <appdomain>.[net/rest/media](#)).

```

1 {
2   "description": "Vehicle - predominantly green, white",
3   "id": "2C6E2FB761CC554EBDF867ABE254AFC444BAED3E-1762772117674",
4   "source": "vision",
5   "legoSetId": "2C6E2FB761CC554EBDF867ABE254AFC444BAED3E",
6   "tags": [
7     "lego",
8     "vehicle",
9     "green",
10    "white"
11  ],
12  "timestamp": "2025-11-10T10:55:17.674835289Z",
13  "_rid": "PRkDAIs3cAUcBQAAAAAAAA==",
14  "_self": "dbs/PRkDAA==/colls/PRkDAIs3cAS=/docs/PRkDAIs3cAUcBQAAAAAAAA==/",
15  "_etag": "\"0a00fd38-0000-5b00-0000-6911c4950000\"",
16  "_attachments": "attachments/",
17  "_ts": 1762772117
18 }

```



Figure 2. - Media Analysis Example.

4. Testing and Evaluation

The system's performance will be evaluated using Artillery. The goal of this evaluation is to assess the scalability and robustness of the backend when deployed in different Azure regions and to understand the impact of network latency and caching on overall performance. Two environments will be configured, Italy North representing the main development region of the system and Sweden Central representing a geographically distinct region used for comparative analysis.

Each test will follow the same workload model, generated from a YAML configuration file, `load-data.yml`, that simulates sequential operations such as user creation, media upload, lego set creation, auction initialization, and bid placement. Both environments will run identical scenarios to ensure a fair comparison.

The comparison between the two regions using Artillery showed that while both environments successfully handled similar workloads, the *Sweden Central* deployment consistently exhibited higher response times. The average latency in Italy remained around 270–300 ms, whereas Sweden averaged 300–350 ms, with p95 values reaching up to 500 ms due to the cross-region calls to Cosmos DB and storage in Italy. The number of successful responses (HTTP 2xx) was nearly identical, confirming system stability and functional equivalence, but throughput in Sweden was slightly lower ($\approx 3\text{--}4$ req/s vs. $\approx 4\text{--}5$ req/s in Italy). Additionally, the complete test execution in *Italy North* lasted approximately 1 hour and 24 minutes, while the *Sweden Central* run completed in about 7 minutes and 13 seconds, reflecting the faster saturation of the Swedish environment under higher latency conditions. Overall, the data clearly demonstrates the expected performance degradation caused by geographical separation, as the Swedish app had to communicate remotely with backend services hosted in Italy, introducing measurable additional latency while maintaining correct behavior under load.

Beyond the Artillery load testing, complementary evaluations were conducted using Azure monitoring tools, which provided further insight into system performance and resource behavior.

4.1. Web App

The web app in Italy shows excellent, fast, and very stable performance, with an average response time of only 26.29 ms. Unlike Italy, the web app in Sweden is extremely unstable, its average time is 253.11 ms. The graph reveals that the application suffered catastrophic latency spikes, with response times reaching almost 2 minutes, indicating serious performance failures or momentary interruptions.

4.2. Cosmos DB

The data usage results highlight the impact of cross-region communication. While the Italy North deployment consumed about 70 MiB, the Sweden Central instance reached roughly 358 MiB under the same workload. This increase reflects the additional data transfer caused by the Swedish web app accessing databases and storage located in Italy, demonstrating how geographical separation significantly raises bandwidth usage and latency.

The server-side latency results were stable across both regions, averaging 0.89 ms in Italy North and 0.93 ms in Sweden Central. This small difference shows that backend processing performance remained consistent, with the slightly higher latency in Sweden reflecting the extra network hop required for cross-region communication with the database in Italy.

Both regions processed a similar workload, with the Italy North deployment handling about 317000 requests and Sweden Central around 333000. Despite the Swedish app depending on cross-region communication with resources in Italy, overall throughput remained stable, showing that the system scaled effectively under distributed conditions while maintaining reliability and consistent performance.

4.3. Blob Storage

The Blob Storage end-to-end latency results reveal a clear difference between regions. In Italy North, the average latency was approximately 3.19 ms, while in Sweden Central it increased to around 4.96 ms. This rise reflects the additional time required for the Swedish web application to communicate with the Blob Storage service located in Italy. Despite this increase, performance remained stable in both environments, indicating that Azure internal network efficiently handled cross-region data transfers while still demonstrating the expected latency overhead from geographical distance.

4.4. Reddit

The Redis CPU load on both servers is low and does not present any problems. The main issue comes from memory, where Redis in Italy has a large cache and Redis in Sweden has a smaller one, however it shows a clear event of “cache warming”, an intensive data load.

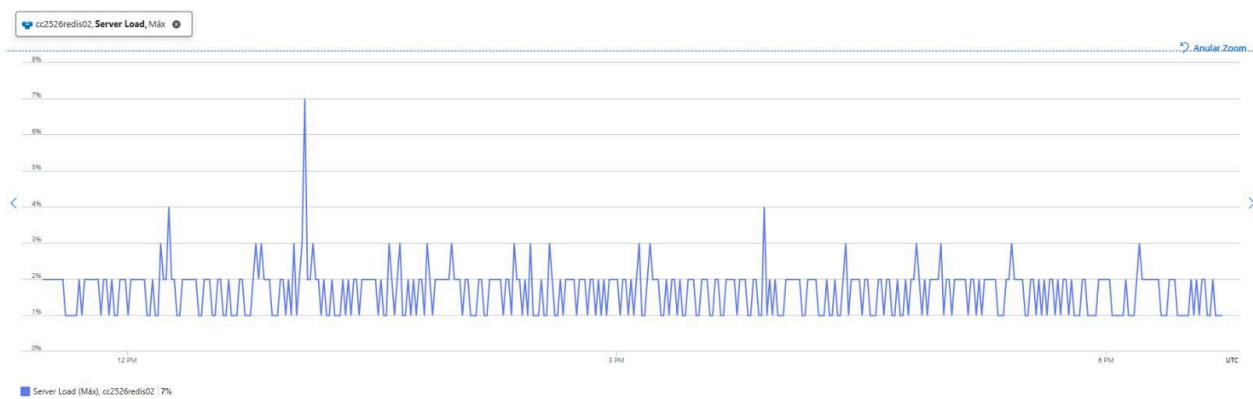


Fig 3. - CPU, Sweden Central.

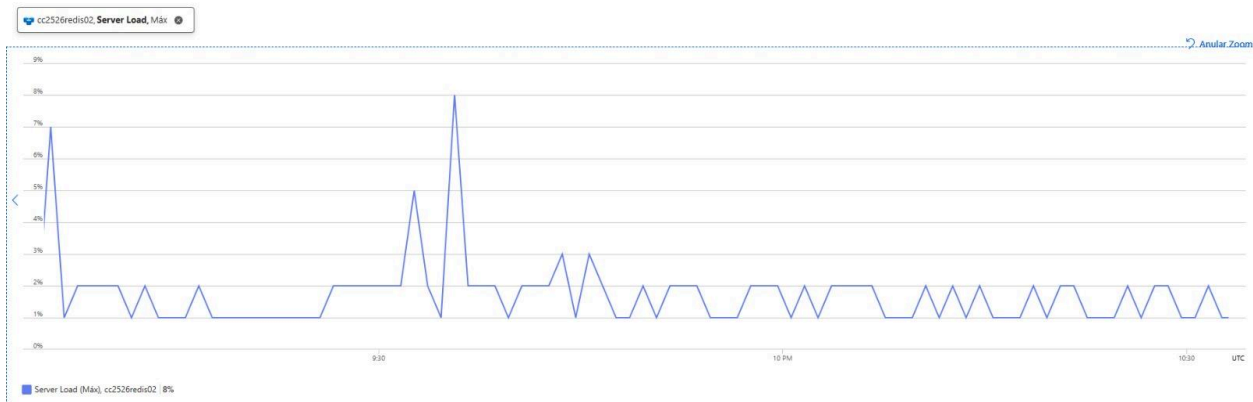


Fig 4. - CPU, Italy North.

5. Conclusion and Future Works

5.1. Conclusion

The developed backend system effectively demonstrates how Azure cloud services can be combined to build a scalable, reliable, and efficient application. By integrating App Service, Cosmos DB, Blob Storage, Redis Cache, and Azure Functions, the architecture ensured modularity, performance optimization, and automated background processing. Performance evaluation with Artillery confirmed the system's robustness across regions. The Italy North deployment achieved lower and more stable response times compared to Sweden Central, where additional latency resulted from cross-region communication. Despite this, both instances maintained correct operation and stable throughput, proving the system's resilience under distributed conditions. Overall, the project met its objectives by implementing a cloud-native backend that leverages caching, asynchronous processing, and intelligent media analysis. The results highlight how well-designed Azure architectures can deliver high availability and scalability, while offering a solid foundation for future extensions such as stronger hashing, advanced analytics, and cost-optimized operations.

5.2. Future Works

Looking ahead, there are several opportunities to enhance the system by extending its intelligence, scalability, and security. The main directions for future work include:

- Enhance computer vision: fine-tune models with larger and more diverse LEGO/non-LEGO datasets (pieces, colors, parts).
- Content moderation: integrate Azure Content Moderator or an NSFW/illegal-content classifier, this could be vinculated with a account ban function.
- Scalable analytics: feed Cosmos DB change feed into Cognitive Search and Spark/Databricks; cache hot results in Redis.
- Operations and privacy: enforce authentication, audit logging, rate limits, and data-retention policies.
- Upgrade the Hashing Algorithm (SHA-1) to a more secure e.g: SHA-256
- Special Data Visualization Endpoint protection, validation of inputs, [evil-regex protection](#), and set usage costs for azure functions so our wallets don't disappear.